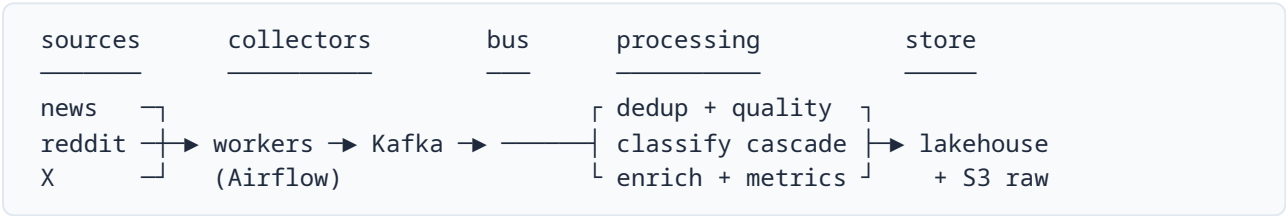
Part 3 — Automating Daily Data Collection

How I would collect mentions from news sites, Reddit, and X every day, store them, keep them clean, and scale it. Part 1's classifier becomes the processing core; this is the system around it. The honest starting point is that the hard part in 2026 is no longer the modelling — it's **access**: the platforms have closed their data behind paid, gated APIs, so most of the design effort goes into collection and cost control.

Architecture

Start with **daily batch**; it is cheaper, simpler, and enough for reputation reporting. Move to a **streaming-first lakehouse** only for the parts that need minutes, not hours (crisis detection on X). That pattern — Kafka for transport, Flink for stateful processing, Iceberg for storage, Debezium CDC to keep the warehouse current — is what's replacing nightly batch ETL in 2026, so it's the right thing to grow into rather than build day one.



Collecting

Each source gets a thin adapter that maps its payload to one schema (`{id, date, source, source_tier, url, title, text, channel, reach?}`), so the rest of the system never sees source-specific shapes. A brand-match check (the relevance filter from Part 1, as entity resolution) runs at the edge so obvious noise never enters the bus.

News. Cheapest and most reliable first: RSS/Atom and sitemaps, plus Google News RSS and GDELT for discovery and a news API (NewsAPI) as a backstop. Scraping is the last resort — Scrapy, upgraded to Playwright for JavaScript pages, or a managed scraper (Apify, Bright Data) when sites fight back with anti-bot. A commercial aggregator (Meltwater, Brandwatch) is the buy-instead option.

Reddit. The official Data API is approval-gated and contract-priced (around $12k/yr for the standard tier; the free OAuth tier is ~100 requests/minute), and Pushshift — the old archive everyone used — lost access in 2023. The production pattern now is hybrid: the official API for the live hot path, and a pay-per-call provider or Apify for backfill and overflow, merged in the warehouse. PRAW is fine for the official side.

X / Twitter. This is the expensive one. X moved to pay-per-use (about $0.005 per post read, capped at 2M reads/month) with Enterprise around $42k/month; the old $200 Basic and $5k Pro tiers are closed to new users. Third-party providers (TwitterAPI.io, Apify, Bright Data) run roughly 30× cheaper per tweet with historical access, so unless first-party compliance is a hard requirement, they're the pragmatic choice. Use keyword and cashtag rules on the filtered-stream / recent-search endpoint.

A scheduler (Airflow or a cloud equivalent) runs the collectors daily, continuously for X.

Storing

Three layers, each with one job:

- **Raw landing zone (S3).** Every payload stored as-is, partitioned by source and date. It's the audit trail and it makes the pipeline replayable — re-run classification without re-scraping.
- **Lakehouse / operational store.** Apache Iceberg (or Delta) for analytical history at scale; a Postgres store with a `pgvector` column works below that volume and gives you embeddings for dedup and semantic search in the same place. Partition by date, upsert by content key.
- **Warehouse (BigQuery / Snowflake).** Aggregates that serve the dashboard and BI once volume outgrows the operational store.

Every label carries the model and prompt version, so results are reproducible and a model change can be back-filled cleanly.

Keeping it clean

Deduplication is a three-stage funnel — the same three passes from Part 1, swapped for algorithms that hold up at scale:

1. **Exact** — canonicalise the URL and hash the content.
2. **Near-duplicate text** — MinHash + LSH (locality-sensitive hashing), the standard for large-scale near-dup detection: it approximates Jaccard similarity in sublinear time, so it stays cheap as the archive grows. Run it against a rolling window, not all history.
3. **Semantic / cross-source syndication** — embeddings plus approximate nearest-neighbour search (FAISS, `pgvector`, or Milvus) to catch the same story reworded across outlets.

Upserting by content key makes re-runs idempotent. Before anything is classified it passes quality gates: schema contracts (Great Expectations or Pydantic), language detection, dead-link checks, spam and bot heuristics (account age, repost rate), and the brand-relevance filter. **PII is masked before any text reaches an external model.** Confidence and sentiment distributions are tracked over time, so model or data drift shows up as a measurable shift rather than a surprise.

Scaling and cost

The one decision that keeps this affordable is the **classification cascade** from Part 1: the cheap local model handles the confident majority and only low-confidence records reach the LLM, which caps cost as volume grows. Batch the escalated calls and cache prompts to push it further.

The rest is standard. A queue between collection and processing lets the two sides scale independently, so a slow scraper never blocks classification. Collectors and classifiers run as autoscaling stateless containers. Each source manages its own rate limits with backoff. Processing is incremental — only new or changed records each cycle — and CDC propagates updates without re-scanning the archive.

Build vs. buy

A managed social-listening platform (Brandwatch, Meltwater, Sprinklr) gets you live in days but is a black box: you don't control the taxonomy, and you pay enterprise rates. Building gives full control of the driver/sub-driver framework and the cascade economics. The sensible split is **hybrid — buy the collection** (the brittle, gated, constantly-breaking part) **and build the classification and intelligence** (the actual differentiator, and where Part 1 already does the work).